Math Practice Web Application

Development Report

Live application: https://math-practice-app-nebraassattar.netlify.app
GitHub repository: https://github.com/nebraassattar/math-practice-app

Approach

The goal was to build an arithmetic practice application for children ages 7-12 using React, TypeScript, and Vite. The application supports addition, subtraction, multiplication, and division at four difficulty levels. Users select their practice settings, choose an answer, reveal the solution, and report whether they got the question right or wrong. Results, preferences, and mistake patterns are saved in the browser. Mistake-review sessions generate related questions and track progress toward mastery.

Development followed the phases in `TODO.md`, with `specifications.md` defining expected behavior. This made the work manageable and provided checkpoints for reviewing AI-generated changes. Early phases established the project, shared components, routing, and practice configuration. Later phases added question generation, sessions, statistics, persistence, mistake review, accessibility, recovery, and deployment. Related phases were sometimes combined when implementing them together made the behavior easier to test, such as question generation and answer choices.

The workflow alternated between implementation, automated checks, and human review. The AI inspected the repository, implemented a bounded set of tasks, added tests, and summarized the changes. I then exercised the application and reviewed its behavior before approving commits and the next phase. Correcting the reference documents early was an important step: a file intended to contain the software specification still contained a duplicate checklist. Implementation paused until the actual specification was available.

The code separates interface components, page-level flows, services, types, and utilities. Arithmetic rules and session transitions are separate from the visual question card. Storage validation and migration are handled through a storage service. This structure supported testing the underlying logic without requiring every test to interact with the browser. Netlify hosts the built application, with a fallback rewrite that allows React Router routes to load directly or after a refresh.

Effective Prompts

Three prompts illustrate how specific instructions improved the collaboration. These are excerpts from the development conversation.

1. Establishing context before making changes:

> “Read specification.md, TODO.md, README.md, and the existing source code. Also check Git status and recent commits.”
> “Run the existing tests, lint, formatting check, and production build. Do not modify files yet.”

This prompt required investigation before implementation. It helped establish what already existed and whether the current baseline worked. The specification file was subsequently standardized as `specifications.md`.

2. Defining a bounded implementation:

> “Implement only Phase 3: Practice Configuration, tasks 7-10.”
> “Pass the validated configuration to /practice/session. That page should temporarily display the chosen operations, difficulty, and session length so we can verify the handoff.”
> “Do not generate questions, implement sessions, add persistence, or implement mistake review yet.”

This was effective because it specified both the required result and the boundaries. The temporary configuration display made the handoff observable before session behavior was added. It also prevented later features from obscuring problems in setup.

3. Making deployment requirements explicit:

> “Add or update netlify.toml in the repository root”
> “A single-page application fallback redirect from /* to /index.html with status 200, so React Router routes work when opened directly or refreshed.”
> “Do not commit, push, or deploy yet, and do not mark the application complete if required features remain unfinished.”

This prompt turned deployment into a reviewable configuration task. It required checking Node compatibility and running the project checks before publishing. Separating preparation from publication preserved control over changes to GitHub and the live site.

Debugging Example

One meaningful regression appeared during performance optimization. An optimization skipped saving settings when the submitted settings matched the settings already in memory. This avoided unnecessary storage writes, but initially returned success before checking whether the stored data belonged to a newer, protected schema version.

The existing persistence test caught the problem. The raw newer-version data remained unchanged, but the expected save-protection warning was not produced. This showed that preserving data alone was insufficient: the application also needed to accurately report that it could not save changes.

The fix limited the unchanged-settings shortcut to cases where storage was not in read-only protection mode. Protected data continued through the normal warning path. The full suite was rerun afterward and passed. This example demonstrated the value of testing user-visible behavior alongside data integrity, especially when an optimization changes the order of validation and saving.

Testing and Verification

The final verified suite contained **319 tests across 20 files**. Unit tests covered arithmetic boundaries, division validity, distractor uniqueness, statistics, mistake-pattern updates, related-question generation, mastery, storage validation, and session recovery. Component and integration tests covered configuration, answer selection, card revealing, both self-report controls, session completion, resets, and navigation confirmation.

All tests, ESLint, formatting checks, and the TypeScript/production build passed. A keyboard-only test completed a five-question session. Axe accessibility scans found no violations in the tested jsdom states, but those scans did not verify rendered contrast, reflow, or actual screen-reader speech.

Browser checks also tested the deployed site. Direct routes and refreshes worked, unknown URLs showed the application fallback, and Back/Forward navigation worked. A five-question live session retained its revealed card after refresh and saved the expected statistics. Mobile checks found no horizontal overflow on the tested pages at 320 pixels. A separate 5,000-attempt test preserved lifetime totals while limiting recent-session details to 100 records.

Remaining checks are documented rather than treated as passes. These include real screen-reader testing, additional named browsers and devices, slower-device performance, and follow-up on one console error during rapid refresh testing that did not reproduce on subsequent checks.

AI Strengths and Limitations

AI was useful for implementing repetitive interface patterns, extracting reusable services, generating regression tests, and explaining failures. It also helped connect changes across routing, storage, configuration, and documentation. Explicit phase boundaries and acceptance criteria made its output easier to inspect.

However, generated code still required verification. The settings optimization introduced a regression despite having a reasonable performance goal. Automated accessibility scans could not establish complete accessibility, and the available browser environment could not substitute for every device or screen reader. Keeping incomplete checks visible was therefore essential.

The project showed that AI assistance works best as an iterative development process: provide reliable requirements, constrain each change, inspect the result, run meaningful tests, and review the application directly. Human judgment remained necessary for usability decisions, accepting the deployed interface, and distinguishing delivered features from checks that were still outstanding.